Four Further Lean Proof Refinements

An audited continuation through proposal rounds a–v

Prepared for Vladimir Reshetnikov

26 October 2026

Repository snapshot: `1e2a853935a6155cbcf2aa199aaca51946c4fff9`
Closing source/novelty refresh: `f42d3f94f78eb63639316a33e25f2f69d12c651e`
Lean `4.32.0`; mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`

Abstract

This review proposes four concrete changes to the maintained Lean proofs: derive formula-code lower bounds from outer-constructor minima; obtain strict multiplication and the Boolean idempotent law without successor discreteness; replace two bilateral integer inductions by period-one rigidity; and expose the exact canonical band quotient before hiding it in an existential. The proposals target seven existing source files. They were screened against the eighteen initially archived simplification rounds, substantive notes and prototypes, the three initially pending ZIPs, and the incoming directory's index. A closing refresh checked the later rounds s–v as well. Every substantive earlier proposal is treated as available, irrespective of its adoption or validation status. The contribution is the particular residual proof reduction in each target, rather than a claim that monotonicity, cancellation, periodicity, or canonical witnesses are new mathematical techniques. The packet supplies exact patches, replacement modules, standalone proof kernels, provenance, and validation receipts. Source-size measurements charge the new helpers and imports as well as deleted proof text.

Contents

1 Result, scope, and recommended order **3**

1.1 What the measurements mean . . . 3

1.2 Measured source changes . . . 4

2 Exclusion audit and limits of the novelty claim **4**

2.1 Ideas deliberately excluded . . . 4

2.2 Nearest precedents for the retained proposals . . . 5

3 Proposal A: derive the formula threshold from constructor minima **6**

3.1 The current obligation . . . 6

3.2 The mathematical observation . . . 6

3.3 Lean implementation . . . 6

3.4 Why this remains useful after the earlier refactors . . . 7

3.5 Compatibility and validation target . . . 7

4 Proposal B: prove the algebra before successor discreteness 7

4.1 The dependency in the current proof . . . 7

4.2 Strict multiplication from a nonzero additive difference . . . 8

4.3 The Boolean-root theorem is then library reuse . . . 8

4.4 The two-file dependency reversal . . . 9

4.5 Why the result is stronger than a reorganization suggestion . . . 9

5 Proposal C: expose period-one functions 9

5.1 A helix is a constant integer orbit . . . 9

5.2 The same mechanism removes seam-time induction . . . 10

5.3 Why this is not the old centered-induction proposal . . . 10

6 Proposal D: preserve the exact witness before existential erasure 11

6.1 Where the duplication comes from . . . 11

6.2 The stronger intermediate statement . . . 11

6.3 The two clients . . . 11

6.4 Layering and cost . . . 12

7 Validation and reproducibility 12

7.1 Meaningful checks rather than duplicated implementation tests . . . 13

7.2 Replay procedure . . . 13

8 Tradeoffs and integration decisions 14

8.1 A: small monotonicity API or private helpers . . . 14

8.2 B: public imports are part of integration . . . 14

8.3 C: use the smallest existing API . . . 14

8.4 D: charge the new interface honestly . . . 14

9 Further research questions 14

10 Source references 15

A Archive guide 16

1 Result, scope, and recommended order

The strongest conceptual reduction is the internal arithmetic change: successor decomposition is unnecessary for the multiplication-order law used to obtain cancellation. Once strict multiplication is available earlier, the Boolean-root theorem becomes an application of the existing idempotent theorem. The most direct low-risk change is the formula-code bound: its recursive hypotheses were stronger than needed, because the outer constructor already forces the numeric code above the required threshold.

The two Wang-tile changes concern recently added proofs. Reparameterizing a helix gives an integer function of period one, so a library theorem replaces the positive- and negative-time induction branches. Separately, two completeness proofs reconstruct the same extended-band quotient. Proving the matching and binding properties for that exact expression first makes the two callers short wrappers, while retaining the literal canonical word used by the marker theorem.

ID	Proposal	Main target	Main benefit
A	Constructor minima	`AnnotatedClosure.formCode_ge`	Removes structural induction and square-bound helpers
B	Cancellation before discreteness	`ObjectNaturalDiscrete.mul_self_eq_self_iff`	Removes a successor-based algebraic proof
C	Period-one rigidity	Helical row-zero and seam constancy	Removes two bilateral inductions
D	Exact band quotient first	CycleBand and BandMarker	Removes a duplicated construction proof

For integration, apply A and C first: they are local proof replacements. Apply B as its two-file patch, since its import reversal is part of the argument. Apply D as its two-file patch, since the later marker theorem consumes the new lower-level theorem. The patches are independent at the file level except for the ordinary existing dependency of band construction on the helical quotient. The archive includes a combined patch for convenient review.

1.1 What the measurements mean

The code was compared with one immutable repository snapshot. The user asks us to assume all earlier proposals have already been applied, but those proposals do not constitute one mechanically merged alternate branch: some provide alternatives or design sketches. Accordingly, the *novelty baseline* treats all their mechanisms as available, whereas the *measurement baseline* is the pinned current source. The measured line savings are local deltas, not a forecast of an unseen fully refactored tree. Where earlier structural refactors change surrounding code, the new proof kernel should be transplanted into the corresponding residual obligation.

No change in theorem strength is being substituted for a shorter proof. The existing public theorem statements are retained in the supplied overlays. Proposal D adds one stronger, exact-witness theorem. The program definitions and numeric coding functions used in proposal A are unchanged; proposal D leaves the existing `canonicalWord` definition unchanged.

1.2 Measured source changes

Proposal / source files	Before	After	Reduction
A: AnnotatedClosure	148	129	19
B: OrderedSemiring + Discrete	103	90	13
C: HelicalQuotient + SeamColumns	253	238	15
D: CycleBand + BandMarker	291	287	4
Total, seven complete files	795	744	51

Counts are physical lines of complete source files, including imports, statements, comments, blank lines, and newly introduced helpers. They are not proof-term-size measurements. The aggregate reduction is approximately 6.4% on these seven files. Proposal B's strictness proof is not shorter by itself; it enables the later cancellation-based deletion. Proposal D's interface and documentation costs are included.

2 Exclusion audit and limits of the novelty claim

The source snapshot contains archived rounds `2026-10-26a` through `2026-10-26r`. We checked their main manuscripts, substantive supplementary notes, cumulative exclusion ledgers, and candidate-related Lean prototypes and patches. A selected corpus of 488 files was verified against the pinned Git blob hashes; its SHA-256 manifest is included. Verification of bytes and inspection of meaning are distinct: this does not claim that every incidental unchanged source snapshot was independently analyzed as a proposal.

The three ZIPs present in `docs/incoming` at the snapshot were also extracted and reviewed:

- `diophantine-lean-further-simplifications-2026-10-26.zip`;
- `diophantine-lean-further-simplifications-2026-10-26(1).zip`;
- `diophantine-lean-structural-refinements-2026-10-26.zip`.

The incoming index was searched across the full document and its relevant current and historical proof-simplification summaries were inspected. References in that index to older external dossiers are not a claim that every historical linked dossier was retrieved anew.

A closing refresh found that main had advanced to `f42d3f94f78eb63639316a33e25f2f69d12c651e`. The three pending archives had become rounds s, t, and u, and round v had arrived. Their placement records and substantive content were checked. The original a–r directory hashes were unchanged, and all seven target source files were byte-identical to the validated snapshot. Round v adds last-slot Option/Fin encoding, action-halt equality reflection, a write-site-only split, an absolute-value trace-bound conversion, translated guarded interval induction, and shared west/east bond pulses. None is the mechanism proposed here. This closing check is a novelty and target-source refresh; the full compiler replay remains pinned to the original dependency snapshot.

2.1 Ideas deliberately excluded

The old packets already cover a wide range of useful refactors. Recommending them again, even with a different helper name or a new compilation log, would not satisfy the request. In particular, this report does not count the following as new proposals:

- Recursor congruence, assignment naturality, witness reindexing, exact numeric checker interfaces, list/Option API reuse, and scanner simulation/fusion.
- Shared sums of residual squares, semiring gate semantics, DAG induction, indexed power certificates, sparse row reflection, and common circuit tails.
- Carry certificates, finite-fiber sum identities, aligned-cell coordinates, packed-direction records, binary-word congruence, physical-selector interfaces, and same-word selection coherence.
- Pell recurrence comparisons, scaled error estimates, signed congruence transport, rational-root descent, exact quotient rounding, and minimal fundamental-unit arguments.
- Internal satisfaction normalization, definable-induction wrappers, shared certificate closure, and a generic internal expression evaluator.
- Centered interval induction, first-exit arguments, and shared-edge Wang rows from the pending archives.

The complete exclusion notes are supplied in `audit/`; their purpose is to make the screening reviewable, not to inflate the contribution count.

2.2 Nearest precedents for the retained proposals

A: constructor minima.
Earlier work on `AnnotatedClosure` concerns typed/numeric iteration fusion. Previous pairing proposals concern inverse graphs and canonical witnesses. The present change removes the recursive lower-bound proof by using only coordinate monotonicity and outer-constructor minima.

B: cancellation before discreteness.
Round c explicitly noticed the `OrderedSemiring->Discrete` dependency and proposed weaker algebraic lemmas or module reordering. We do not claim that observation again. The new concrete step is a proof of strict multiplication from an additive difference and `NoZeroDivisors`, followed by the exact existing idempotent theorem. The mathematical hypotheses, rather than module organization alone, justify the reversal.

C: period-one rigidity.
Generic cyclic geometry and centered integer induction are old. Here the actual helix is reparameterized to expose a function of period one, and `Function.Periodic.int_mul_eq` discharges the entire signed-time argument.

D: exact quotient reuse.
Generic canonical witnesses and source-window sharing are old. Here a specific repeated `extensio n_properties`/quotient/band-binding pipeline in the new CycleBand and BandMarker proofs is factored while preserving the exact `canonicalWord` expression.

These are scoped non-repetition claims against the inspected repository corpus. They are not claims of originality in mathematics or in the broader Lean ecosystem.

3 Proposal A: derive the formula threshold from constructor minima

3.1 The current obligation

`AnnotatedClosure` proves that every valid formula has a numeric code at least 44. This bound is subsequently used to place the universal-quantifier encoding in the correct branch of square pairing and derive

$$\mathrm{formCode}(\forall f) = \mathrm{formCode}(f)^2 + 39.$$

The threshold matters because the numeric code is used by literal primitive-recursive programs; replacing the code by an equivalent encoding would be a different change.

The original proof establishes auxiliary square bounds by unfolding `Nat.pair` and splitting its branches. It then performs structural induction on formulas, carrying lower-bound hypotheses for the subformulas. Those hypotheses are unnecessary for the requested threshold.

3.2 The mathematical observation

Write the actual coding combinators as

$$A(n) = \mathrm{pair}(0, n) + 1, \qquad F(a, b) = \mathrm{pair}(a + 1, b) + 1.$$

Mathlib supplies strict monotonicity of `Nat.pair` in each coordinate. Consequently, F is monotone in both coordinates, and $A(n) \geq 1$.

A variable-bearing atomic code has the form

$$F(A(t), F(A(i), A(j))).$$

It therefore satisfies

$$F(A(t), F(A(i), A(j))) \geq F(1, F(1, 1)) = 67 > 44.$$

For every connective tag $t \geq 3$, any natural payload p, including zero, satisfies

$$F(A(t), p) \geq F(A(3), 0) = 133 > 44.$$

The bottom constructor is checked directly from its existing definition. Thus a case split on the *outer* constructor proves the bound. There is no reason to inspect the subtree codes.

The two numeric minima above are only convenient witnesses above 44. The patch does not assert that 67 or 133 is the sharp minimum of every individual syntactic family. It also does not change the published threshold.

3.3 Lean implementation

A small private monotonicity lemma composes the two existing library facts:

```
private theorem pair_mono {a a' b b' : Nat}
    (ha : a <= a') (hb : b <= b') :
    Nat.pair a b <= Nat.pair a' b' :=
  ((show StrictMono (fun a => Nat.pair a b) from
    fun _ _ h => Nat.pair_lt_pair_left b h).monotone ha).trans
      ((show StrictMono (Nat.pair a') from
        fun _ _ h => Nat.pair_lt_pair_right a' h).monotone hb)
```

Together with the elementary `atom_pos` bound, the derived `fork_mono`, `variable_min`, and `connective_min` lemmas provide the complete local infrastructure. The main theorem becomes a constructor case split:

```
cases f with
| fMem i j => exact variable_min 0 i j
| fEq i j => exact variable_min 1 i j
| fBot => decide
| fImp a b => exact connective_min 3 _ (by decide)
| fAnd a b => exact connective_min 4 _ (by decide)
| fOr a b => exact connective_min 5 _ (by decide)
| fAll a => exact connective_min 6 _ (by decide)
| fEx a => exact connective_min 7 _ (by decide)
```

The small `decide` calls prove closed constant comparisons. They do not replace a symbolic theorem by testing a finite sample of formulas. All formulas remain quantified in the theorem, and the kernel checks the constructor proof.

3.4 Why this remains useful after the earlier refactors

An exact checker interface, a generic scanner simulation, or a shared pairing inverse does not by itself eliminate this structural induction. The new fact is that the threshold is a property of the outer code constructor. If a prior refactor has already introduced monotonicity of `forkCode`, reuse it and omit the private helper; the induction-free proof remains the residual contribution.

The replacement is especially useful for maintenance of the annotation tags: the proof now exposes which tag ranges and constant minima matter. A future encoding change must recheck those minima. It no longer requires someone to understand an induction whose hypotheses happen to be redundant.

3.5 Compatibility and validation target

The exact `atomCode`, `forkCode`, `formCode`, `closeCode`, `program`, and `quadraticProgram` definitions are retained. The later program-correctness and node-count declarations remain in the compiled module. The archive supplies both a standalone arithmetic kernel and the complete source replacement, so the distinction between checking the numerical mechanism and checking the actual repository bindings is explicit.

4 Proposal B: prove the algebra before successor discreteness

4.1 The dependency in the current proof

The current `ObjectNaturalOrderedSemiring` imports `ObjectNaturalDiscrete`. Its private multiplication-order proof decomposes both a positive multiplier and a positive difference as successor expressions. Later algebraic results, including multiplicative cancellation, consequently arrive only after the discrete-order layer.

The current Boolean-root theorem proves

$$x^2 = x \quad \Longleftrightarrow \quad x = 0 \ \vee \ x = 1$$

by another zero-or-successor decomposition. The proof is valid, but the successor information is unnecessary once a stronger available algebraic interface is used.

4.2 Strict multiplication from a nonzero additive difference

Proposition 1. *Let R be a commutative semiring with a linear canonical additive order, ordered additive cancellation, and no zero divisors. If $0 < a$ and $b < c$, then $ab < ac$.*

Proof. Since $b \le c$, canonical additive order supplies d with $c = b + d$. If $d = 0$, then $b = c$, contradicting $b < c$. Thus $d \neq 0$. The multiplier a is nonzero, so the absence of zero divisors implies $ad \neq 0$. In a canonical order, nonzero elements are positive, hence $0 < ad$. Distributivity and additive order give

$$ac = a(b + d) = ab + ad > ab.$$

No expression of a or d as a successor was used. □

The existing internal natural numbers have the required canonical-order and zero-divisor instances at the earlier layer. The replacement proof is:

```
obtain ⟨d, hd⟩ := exists_add_of_le hbc.le
have hd0 : d ≠ 0 := by
  intro hz
  apply hbc.ne
  simpa only [hz, add_zero] using hd.symm
rw [hd, mul_add]
exact lt_add_of_pos_right _
  (pos_iff_ne_zero.mpr (mul_ne_zero ha.ne' hd0))
```

This proof is at approximately the same local length as the old strictness proof. Its benefit is the weaker dependency. The instance can be established before `ObjectNaturalDiscrete`, so that the discrete module can use the resulting cancellation infrastructure.

4.3 The Boolean-root theorem is then library reuse

In a monoid with zero and cancellation away from zero, an idempotent is zero or one. Algebraically, if $x \neq 0$, then

$$xx = x = x1$$

and cancellation yields $x = 1$. Mathlib exposes the exact equivalence as `IsIdempotentElem.iff_eq_zero_or_one`. The repository theorem therefore becomes:

```
theorem mul_self_eq_self_iff (x : N H) :
    x*x=x ↔ x=0 ∨ x=1 :=
  IsIdempotentElem.iff_eq_zero_or_one
```

The relevant distinction is between an arbitrary semiring, which can have nontrivial idempotents, and the cancellative structure available here. The old comment correctly warns that arbitrary semiring algebra is insufficient. The revised comment states the precise sufficient property: multiplicative cancellation. It does not claim that `NoZeroDivisors` alone, without the order argument, gives cancellation in every semiring.

4.4 The two-file dependency reversal

The patch changes the imports as follows:

Module	Before	After
`ObjectNaturalOrderedSemiring`	Imports `ObjectNaturalDiscrete`	Imports `ObjectResidualSquares` and the canonical-order API
`ObjectNaturalDiscrete`	Imports `ObjectResidualSquares`	Imports `ObjectNaturalOrderedSemiring` and the idempotent API

The earlier module already provides the canonical additive-order and no-zero-divisor facts used by the new proof. The standalone theorem makes those assumptions explicit, while the production patch uses the existing internal-natural instances.

The import change has a real integration consequence: an external file that imported only `ObjectNaturalOrderedSemiring` and relied on its transitive import of discrete lemmas may need an explicit `ObjectNaturalDiscrete` import. The maintained direct consumers were inspected. The umbrella imports Discrete explicitly; `ObjectPairingInjective` retains access through its existing pairing dependency chain. This is why the validation includes downstream pairing modules, rather than stopping after the one-line theorem.

4.5 Why the result is stronger than a reorganization suggestion

Round c already identified this ordering/discreteness cycle. Moving imports without a mathematical replacement would repeat that suggestion and could create a cycle. Here the proof that multiplication preserves strict order has been changed so that it does not consume discreteness in the first place. The dependency reversal is a consequence of the new proof. The reduction also gives a useful hypothesis check: it applies in canonical ordered semirings with non-discrete order, such as nonnegative rationals, so successor decomposition cannot be essential to this argument.

5 Proposal C: expose period-one functions

5.1 A helix is a constant integer orbit

In `WangTiles942HelicalQuotient`, the hypothesis is

$$T(x + w, t - 1) = T(x, t) \qquad (x, t \in \mathbb{Z}).$$

The desired conclusion is

$$T(x, t) = T(x + wt, 0).$$

The current proof performs integer induction on t, generalizing x, with separate successor and predecessor branches. Each branch transports the helix equation and normalizes an affine expression.

For fixed x, t, define

$$f(s) = T(x + ws, t - s), \qquad s \in \mathbb{Z}.$$

Then

$$\begin{aligned} f(s+1) &= T(x + w(s+1), t - (s+1)) \\ &= T((x + ws) + w, (t - s) - 1) \\ &= T(x + ws, t - s) = f(s). \end{aligned}$$

Thus f has period one. The integer-multiple periodicity theorem gives $f(t) = f(0)$. After simplifying the endpoints, this is exactly the desired equality in reverse order.

The checked replacement is:

```
have hp : Function.Periodic
    (fun s : Int => T (x+w*s) (t-s)) 1 := by
  intro s
  simpa only [mul_add, mul_one, <- add_assoc,
    sub_add_eq_sub_sub] using hhelix (x+w*s) (t-s)
simpa using (hp.int_mul_eq t).symm
```

This is an instance of a library theorem that already handles all integer multiples. Negative times are not excluded, and no positivity requirement on w appears. The theorem remains polymorphic in the target type of T; it does not need tiles, finiteness, or a global rectangular period.

5.2 The same mechanism removes seam-time induction

`WangTiles942SeamColumns` already has

$$\mathrm{SeamAt}(T, x, u+1) \iff \mathrm{SeamAt}(T, x, u).$$

Treat $u \mapsto \mathrm{SeamAt}(T, x, u)$ as a proposition-valued function. Propositional extensionality converts the local iff to equality, yielding period one:

```
have hp : Function.Periodic (SeamAt T x) 1 :=
  fun u => propext (seam_successor_iff hAllowed hTime x u)
simpa only [Int.cast_id, mul_one, eq_iff_iff]
  using hp.int_mul_eq t
```

The conclusion is the existing `seam_iff_zero`. No global periodic-plane assumption is introduced. The local transition equivalence itself forces the proposition to be constant over integer time.

The seam proof makes `propext` explicit. That is an axiom-accounting issue worth checking, even when the imported local-transition proof already carries the same dependency. The packet includes before/after axiom output; the report does not infer unchanged axiom sets merely from successful compilation.

5.3 Why this is not the old centered-induction proposal

The pending structural archive contains a useful centered integer-induction framework. It still organizes a two-sided inductive proof. The present proof identifies a concrete orbit function whose period is one, then uses an existing periodicity API. There is no new induction framework or coordinate type. The distinction is mathematical: a translation law on the original two-variable function is converted into constancy along its integer orbit.

The same idea should be used selectively. A transition with a nontrivial output transformation, an inequality instead of equality, or hypotheses valid only on a bounded interval need not define a period-one function. Such cases should continue to use the appropriate induction or interval argument.

6 Proposal D: preserve the exact witness before existential erasure

6.1 Where the duplication comes from

The current `WangTiles942CycleBand` proves that an actual halting run gives a finite marked word. Its proof constructs an extended canonical band, proves the extension's horizontal/temporal compatibility, applies the helical quotient theorem, and restricts the quotient binding back to the original band. It then hides the resulting word under an existential quantifier.

Later, `WangTiles942BandMarker` needs the very same quotient expression, named `canonicalWord`, because its marker-uniqueness theorem is about that literal word. The existential theorem alone does not identify its witness with `canonicalWord`. The later proof therefore repeats the extension, quotient, and band-binding pipeline.

This is not primarily a missing equality simplifier. The earlier theorem exported less information than its proof established.

6.2 The stronger intermediate statement

For the same dimensions and halting hypotheses, let

$$a := \mathrm{quotientWord}\big(\mathrm{extend}(\mathrm{band}(L, H), L + R, L)\big),$$

with the exact existing allowed-tile proof supplied as in the source. The new theorem `band_quotient_properties` proves

$$\mathrm{MarkedWord}((L + R)H, L + R, a)$$

and, for every band coordinate $-L \le x < R$ and every integer time t,

$$\mathrm{tile}(a(\mathrm{address}((L + R)H, L + R, x, t))) = \mathrm{band}(L, H, x, t).$$

The Lean result begins with a `leta:=` containing the actual quotient expression. It does not introduce a new coding, a chosen extensionally equivalent word, or a second canonical construction.

The new theorem retains the real hypotheses: the actual halting state at time T; margins $L, R \ge T + 2$; height $H \ge T + 2$ and $H \ge 3$; positive width; and the existing nonzero typeclass instances for the finite moduli. The wrapper supplies those instances from the same arithmetic hypotheses as before.

6.3 The two clients

The old existential theorem reduces to its existing instance setup followed by

```
exact ⟨_, band_quotient_properties T L R H
  hh hL hR hTH hH (by omega)⟩
```

The marker theorem takes the shared matching/binding pair and adds its own marker characterization:

```
obtain ⟨hm, hbind⟩ :=
  band_quotient_properties T L R H hh hL hR hTH hH hw
exact ⟨hm, canonicalWord_marker_iff T L R H hh hTH
  (by omega) (by omega) hw, hbind⟩
```

This reuse is legitimate because the result is about the literal expression that `canonicalWord` unfolds to. An arbitrary existential witness would not suffice.

6.4 Layering and cost

The helper belongs in CycleBand, before the marker layer. Its proof uses extension matching and quotient binding; it must not import marker uniqueness. The later marker theorem supplies the additional property. This keeps the proof dependencies acyclic and prevents the stronger marker conclusion from becoming an implicit premise of existence.

The physical-line saving is modest after charging the helper's full statement and documentation. The value is a single place to maintain the substantial construction argument. This is not presented as a runtime optimization or a large text compression. A future improvement to the extension or quotient theorem now has one proof client at this boundary, followed by small packaging wrappers.

7 Validation and reproducibility

All four proposals were checked with Lean `4.32.0`. The nine external package revisions match the repository's `lake-manifest.json`; mathlib is pinned to `81a5d257c8e410db227a6665ed08f64fea08e997`.

For proposals A and B, the integration closure has **73 successful module records**. Forty-seven prerequisites were reused from this task's fresh compilations after checking source, direct-dependency, and output SHA-256 hashes; 26 additional compilations completed successfully. No repository object from an older task was used. This closure includes all three modified modules and the unchanged downstream `ObjectNatPairing`, `ObjectPairingSyntax`, `ObjectPairingGraph`, `ObjectPairingSurjective`, and `ObjectPairingInjective`.

For proposals C and D, **34 repository modules** were freshly compiled, including four original focused check modules. The candidate build then recompiled nine modules after source/dependency hash invalidation. All four modified production modules and all four existing focused checks passed. Both patches also passed a combined `gitapply--check` against the pinned source bytes.

The before/after axiom reports are identical for the seven audited existing endpoints listed below. Here the standard three are `propext`, `Classical.choice`, and `Quot.sound`; the new exact-witness helper uses exactly these. No audited endpoint contains `sorryAx`.

Audited endpoint	Axioms, unchanged unless new
`AnnotatedClosure.formCode_ge`	Standard three
`ObjectNaturalDiscrete.mul_self_eq_self_iff`	Same three
`WangTiles942HelicalQuotient.row_zero_of_helix`	`propext`, `Quot.sound`
`WangTiles942SeamColumns.seam_iff_zero`	Standard three
`WangTiles942CycleBand.marked_word_of_halt`	Standard three
`WangTiles942BandMarker.canonicalWord_properties`	Standard three
`WangTiles942BandMarker.canonical_unique_word_of_halt`	Standard three
`WangTiles942CycleBand.band_quotient_properties` (new)	Standard three

Standalone kernels check constructor minima, canonical-order strictness/idempotence, and both period-one arguments independently of the large repository import closures. Those smaller checks complement the complete-module checks; they are not used as substitutes for checking the actual source bindings.

The supplied receipts preserve exact commands, source hashes, dependency hashes, successful return codes, and axiom output. Paths in these historical receipts describe the validation environment; `scripts/replay.py` is the portable entry point for a local checkout. The packet includes the final successful receipts and preserves their original paths for provenance.

7.1 Meaningful checks rather than duplicated implementation tests

The validation is designed around concrete risks:

- Full replacement-module compilation checks that abbreviated mathematical proofs elaborate with the actual definitions and existing theorem statements.
- Downstream internal pairing modules check the reversed import dependency and the availability of discrete declarations to maintained consumers.
- Existing Wang check modules exercise the literal construction and its current public interfaces.
- Before/after `#printaxioms` probes check the trust footprint of the changed public results.
- A source-declaration comparison checks preservation of the old public theorem statements and executable definitions.
- Patch applicability is checked against the pinned bytes. It is not assumed from a visually plausible diff.

These are sufficient for the claims made here. They are not a complete build of the entire ProveIt repository, nor a timing study of all transitive clients. Wall-clock receipts are diagnostic and are not evidence of a general speedup.

7.2 Replay procedure

The archive contains one patch per proposal, a combined patch, complete replacement files for review, and standalone proof kernels. The replay script operates on a local checkout of the pinned repository and rebuilds the changed files and their repository import closure into an isolated output directory outside that checkout. It resolves the installed pinned Lean compiler to its actual executable, verifies the third-party package revisions and the toolchain, manifest, and library-configuration hashes, and invokes Lean directly without loading Lake configuration caches. It does not modify the checkout, push, commit, or publish changes.

Before integration into a newer checkout, inspect the source hash manifest and rebase the patches. Copying a complete overlay onto a file that has accumulated other changes would discard those changes. The patch is the preferred integration unit; the overlays are included to make the exact tested result readable without reconstructing the diff.

8 Tradeoffs and integration decisions

8.1 A: small monotonicity API or private helpers

The supplied patch keeps `pair_mono` and `fork_mono` private. That is appropriate when only this proof needs them. If the counterfactual earlier-refactor baseline already supplies an equivalent monotonicity theorem, use it instead of adding another helper. Publicizing a new abstraction should be justified by actual clients, not by the shortness of its statement.

The constructor minima proof removes nonlinear arithmetic reasoning from the target. It does not seek a sharp lower bound for every constructor and does not simplify the closing programs themselves. This narrow scope protects the executable code and its quoted size facts.

8.2 B: public imports are part of integration

The revised arithmetic proof is conceptually cleaner because it separates cancellation from discreteness. Nonetheless, reordering a module changes what is available through transitive imports. The two-file patch and downstream checks are essential. An external consumer may need one explicit import; that is a compatibility consideration, not a logical gap in the proof.

The Boolean-root statement is left in its existing namespace. Moving its name to a different module would create additional migration work without helping the mathematical simplification.

8.3 C: use the smallest existing API

A new action, quotient, or two-dimensional periodicity framework would be excessive for these two proofs. The existing `Function.Periodic` API already expresses the needed fact. The helper is local and the theorem statements remain unchanged. The proposition-valued application deserves an explicit axiom comparison because it uses `propext`; that comparison is supplied rather than silently assumed.

8.4 D: charge the new interface honestly

A strong exact-witness theorem has a longer statement than the existential wrapper. The source-size table includes that cost. The proposal is worthwhile because two current clients need the same construction and one needs exact witness identity. If later refactors already expose that same exact quotient contract, use their theorem and omit this helper; the residual problem has then already been solved.

9 Further research questions

The following questions develop the four mechanisms rather than re-listing the earlier roadmap. They are not counted as additional validated simplifications.

1. **Which numeric syntax bounds depend only on the root constructor?** The code-minimum argument suggests auditing other tag-threshold lemmas for unnecessary recursive hypotheses. A useful result would classify a specific family of code constructors by a fixed numeric lower envelope, while preserving their literal numeric encodings. Such a classification should be adopted only after identifying additional current clients; a general new coding framework is not proposed here.

2. **How weak can the internal cancellation boundary be?** The checked generic theorem uses a convenient linear canonical order. It would be useful to determine whether a smaller combination of order/cancellation classes suffices for the exact internal-natural instance, without increasing instance-search complexity. Any further weakening must still make positivity of a nonzero additive difference explicit.

3. **Which signed-time proofs are constant-orbit arguments?** Search for another concrete recurrence of the form $F(s+1) = F(s)$ after a coordinate change. The useful boundary is whether the transformation exposes period one globally. Bounded validity or a nontrivial transformation on outputs requires a different proof and should not be forced into this pattern.

4. **Can exact-witness contracts simplify additional completeness consumers?** Follow the canonical band quotient forward to any future arithmetic-witness construction. Exporting the exact lower-level object and its pointwise binding may prevent later equality/coherence proofs. This is conditional on finding a real repeated construction; it is not a new general recommendation to package all witnesses in records.

A successful next iteration should retain the same standard: identify a concrete remaining source obligation, compare its mechanism with the full exclusion corpus, and check a replacement against the literal maintained definitions.

10 Source references

All repository references below use the pinned source snapshot; the package reference uses the pinned mathlib revision.

1. ProveIt source snapshot: `https://github.com/VladimirReshetnikov/ProveIt/commit/1e2a853935a6155cbcf2aa199aaca51946c4fff9`.

2. Archived proposal index and adoption ledger: `https://github.com/VladimirReshetnikov/ProveIt/tree/1e2a853935a6155cbcf2aa199aaca51946c4fff9/Computability/DiophantineConsistency/research/lean-proof-simplification`.

3. Incoming proposal index and three archives: `https://github.com/VladimirReshetnikov/ProveIt/tree/1e2a853935a6155cbcf2aa199aaca51946c4fff9/docs/incoming`.

4. Numeric closure proof: `AnnotatedClosure.lean`; actual code definitions: `AnnotatedProofCode.lean`.

5. Internal arithmetic: `ObjectNaturalOrderedSemiring.lean`, `ObjectNaturalDiscrete.lean`, and `ObjectResidualSquares.lean`.

6. Helical and seam proofs: `WangTiles942HelicalQuotient.lean` and `WangTiles942SeamColumns.lean`.

7. Exact band construction: `WangTiles942CycleBand.lean` and `WangTiles942BandMarker.lean`.

8. Mathlib square pairing: `https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Data/Nat/Pairing.lean`.
9. Mathlib integer periodicity: `https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Algebra/Ring/Periodic.lean`.
10. Mathlib idempotents: `https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Algebra/GroupWithZero/Idempotent.lean`.

A Archive guide

`report/`
This manuscript, its PDF, and generated metric/validation inputs.

`patches/`
Four focused patches and one combined patch, with paths relative to the repository root.

`overlay/`
The complete seven replacement modules, preserving repository-relative paths.

`lean/`
Small standalone proof kernels and axiom-audit probes.

`audit/`
Exclusion and independent review notes, the verified prior-corpus manifest, and current-archive provenance.

`evidence/`
Compiler receipts, exact source hashes, public declaration comparison, and before/after axiom output.

`scripts/`
Reproduction and artifact-integrity checks. Read the top-level README for the exact commands.